PROJE 14 / 20 · ÇALIŞAN TEMEL SÜRÜM

Parçalı Dosya Yükleme ve Paylaşım

Dosyaları 256 KB parçalarla yükle; tamamlanan dosyaları bir saatlik bağlantıyla paylaş.

8 MB sınır · süreli paylaşım

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Parça sıra ve boyut kontrolü · Tekrar parça yazımı · Eksik parça engeli · SHA-256 sonuç özeti · Yetkili indirme ve süreli link

Bu sürümde olmayanlar

Klasör ağacı ve önizleme yok. Dosya parçaları SQLite BLOB alanında tutulur; S3 presigned URL değildir. Arayüz yeni yükleme başlatır; tarayıcı yeniden açıldığında yükleme devam ettirme ekranı yok.

02 / Backend ve veri modeli

files yükleme kaydını; chunks (file,part) anahtarı parça sırasını tutar. Complete transaction önce parça sayısını sonra birleştirilmiş boyutu doğrular. Paylaşım tokenının yalnız SHA-256 özeti saklanır. Dosya silininces foreign key cascade parçaları ve paylaşımı kaldırır. İndirme application/octet-stream olarak sunulur.

Gerçek SQL şeması

```
CREATE TABLE files(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),name TEXT,size
INTEGER,total INTEGER,status TEXT DEFAULT 'uploading',created_at TEXT);


 CREATE TABLE chunks(file TEXT REFERENCES files(id) ON DELETE CASCADE,part INTEGER,data
BLOB,PRIMARY KEY(file,part));


 CREATE TABLE shares(token_hash TEXT PRIMARY KEY,file TEXT REFERENCES files(id) ON DELETE
 CASCADE,expires INTEGER);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametreli SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/files	Gövde yok	200
POST /api/uploads	JSON: size, total, name	201
PUT /api/uploads/:id/parts/:part	JSON: base64	200
POST /api/uploads/:id/complete	Gövde yok	200
GET /api/files/:id/download	Gövde yok	200
POST /api/files/:id/share	Gövde yok	200
GET /share/:token	Gövde yok	200
DELETE /api/files/:id	Gövde yok	200

Somut istek örneği

```
POST /api/uploads
{"name":"hello.txt","size":5,"total":1}
201 {"id":"<file-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Eksik parçalı yükleme tamamlanamamalı.
- Aynı parça tekrar gönderildiğinde dosya büyümemeli.
- İndirilen içerik özgün baytlarla aynı olmalı.
- Dosya silindikten sonra paylaşım bağlantısı çalışmamalı.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const f=await post('/uploads',{name:'hello.txt',size:5,total:1},201);
await post('/uploads/'+f.id+'/complete',{},409);
await call('/api/uploads/'+f.id+'/parts/0','PUT',{base64:Buffer.from('hello').toString('
base64')},200);
await call('/api/uploads/'+f.id+'/parts/0','PUT',{base64:Buffer.from('hello').toString('
base64')},200);
await post('/uploads/'+f.id+'/complete',{});
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Küçük bir dosya yükle; Network panelinde upload/part/complete sırasını izle. Eksik parça ile complete isteğinin 409 verdiğini göster.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Yarım yüklemeyi sürdürme ekranı ekle
- 2. Klasör/izin modeli ekle
- 3. Objeyi S3 uyumlu depoya taşı
- 4. Kota ve eski yarım yükleme temizliği ekle

Anlamlı commit sırası

```
git add .
git commit -m "chore: add file-sharing baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Bir parça yeniden gelince neden append yapılmamalı? Hangi noktada checksum doğrulaması daha güçlü olur?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.